

Direct & Hierarchical Control Patterns	Router Multi-Agent	A router reads the request and selects exactly one specialist. Only that specialist answers. Real-world Example: A customer-support desk sends a duplicate payment issue to billing, a login issue to technical support, and a general question to general support. User → Router → One Specialist → Response This is a router-based LLM workflow with specialized LLM nodes.	<pre>graph TD; start([start]) --> router[router]; router -.-> billing[billing]; router -.-> general[general]; router -.-> technical[technical]; billing --> end([end]); general --> end; technical --> end;</pre>
Direct & Hierarchical Control Patterns	Subagent / Agent-as-Tool	A main coordinator remains in control and calls specialist agents as tools. Each specialist has a focused role. Real-world Example: A consultant asks a researcher for facts, a math expert for a calculation, and a writer to present the combined answer clearly. User → Main Agent → Specialist Tools → Main Agent → Final Answer	<pre>graph TD; start([start]) --> model[model]; model -.-> end([end]); model -.-> tools[tools]; tools -.-> model;</pre>
Direct & Hierarchical Control Patterns	Supervisor Multi-Agent	A central supervisor repeatedly selects the next worker, reviews accumulated work, and finishes when enough work is available. A six-turn limit prevents an endless loop. Real-world Example: A project manager assigns research, calculation, and writing tasks one at a time, then creates the final training note. Supervisor → Worker → Supervisor → Another Worker → Finish	<pre>graph TD; start([start]) --> supervisor[supervisor]; supervisor -.-> research_agent[research_agent]; supervisor -.-> math_agent[math_agent]; supervisor -.-> writer_agent[writer_agent]; supervisor -.-> finish[finish]; research_agent --> end([end]); math_agent --> end; writer_agent --> end; finish --> end;</pre>
Direct & Hierarchical Control Patterns	Hierarchical Supervisor	A top supervisor selects one team subgraph. The selected team performs specialist work and then rewrites it as a concise report. Real-world Example: A company director sends a task to either the research department or the business department instead of managing every employee directly. Top Supervisor ┐→ Research Team Subgraph → END └→ Business Team Subgraph → END	<pre>graph TD; start([start]) --> top_supervisor[top_supervisor]; top_supervisor -.-> business_team[business_team]; top_supervisor -.-> research_team[research_team]; business_team --> end([end]); research_team --> end;</pre>
Dynamic & Distributed Handoff Patterns	Handoff Multi-Agent	The triage agent selects a specialist and uses Command(goto=...) to transfer graph control. The selected specialist completes the request. Real-world example: A reception desk transfers a 401 API error to technical support rather than trying to solve it itself. User → Triage ┐→ Billing → END └→ Technical → END	<pre>graph TD; start([start]) --> triage[triage]; triage -.-> billing_agent[billing_agent]; triage -.-> technical_agent[technical_agent]; billing_agent --> end([end]); technical_agent --> end;</pre>
Dynamic & Distributed Handoff Patterns	Swarm Multi-Agent	Agents can hand control directly to another agent. InMemorySaver preserves the active-agent state for the thread. Real-world Example: A math expert calculates a result and hands it to a writer for professional presentation	

		A math expert calculates a result and hands it to a writer for professional presentation. Math Agent $\rightleftharpoons$ Writer Agent Full flow START $\downarrow$ Math Agent $\downarrow$ LLM decides: Can I handle this? $\text{Yes} \rightarrow \text{answer}$ $\text{Need writing} \downarrow$ Handoff Tool $\downarrow$ Writer Agent $\downarrow$ LLM decides: Need math? $\text{Yes} \rightarrow \text{final response}$ $\text{No} \rightarrow \text{final response}$ $\text{Yes} \downarrow$ Handoff Tool $\downarrow$ Math Agent Why is this more clearly multi-agent? Because both agents are actual agents with tools: Math Agent $\rightarrow$ has handoff tool $\rightarrow$ dynamically decides whether to use it Writer Agent $\rightarrow$ has handoff tool $\rightarrow$ dynamically decides whether to use it So unlike a simple routing workflow, the agents themselves decide when control should move. A swarm is a decentralized multi-agent architecture where agents can directly hand off control to one another based on the task, without requiring a permanent central supervisor. And the simple difference: - Supervisor Pattern $\rightarrow$ one central supervisor decides everything - Swarm Pattern $\rightarrow$ agents directly hand off control to each other	
Collaborative & Parallel Processing Patterns	Parallel Multi-Agent Collaboration with Send	A coordinator creates a structured task plan. Send starts one specialist worker per task, the reducer collects all outputs, and the aggregator creates the final answer. Real-world Example: A financial company asks research, security, and business specialists to evaluate an AI-agent proposal at the same time. Coordinator $\rightarrow$ Dynamic Workers $\rightarrow$ Reducer $\rightarrow$ Aggregator $\rightarrow$ END	
Collaborative & Parallel Processing Patterns	Coordinator-Guided Peer Network <i>(It is inspired from Network Agent)</i>	Research, analysis, and writing peers share notes. A lightweight coordinator selects the next peer after each step. This is coordinator-guided, not a pure peer-to-peer network. A six-turn limit prevents an endless loop. Real-world Example: A round-table facilitator decides whether the researcher, analyst, or writer should speak next until the group has a complete explanation. Coordinator $\rightarrow$ Peer $\rightarrow$ Shared Notes $\rightarrow$ Coordinator $\rightarrow$ Next Peer $\rightarrow$ Finish	

*** DEEP AGENT ***

A **Deep Agent** is an advanced class of AI systems engineered to handle complex, multi-step tasks that require sustained reasoning, planning, and execution over extended periods (long-horizon operations). Unlike a standard chatbot or basic single-turn tool-using bot that handles short interaction chains, a deep agent functions more like an autonomous analyst or project manager.

Key characteristics of a Deep Agent include:

- Long-Horizon Running:** They maintain context, coherence, and goal alignment across hundreds of operations spanning minutes, hours, or even days, avoiding context drift and degradation.
- Resource-Intensive Architecture:** Because they continuously plan, reflect, self-monitor, spawn asynchronous sub-agents, and evaluate intermediate steps, they consume significantly more compute, token, and system resources than traditional prompt-response loops.
- Advanced Orchestration (Single or Multi-Agent):** Any single agent or multi-agent topology can act as a deep agent by incorporating foundational architectural pillars:
 - Explicit Planning:** Decomposing high-level goals into granular, trackable to-do lists and milestones.
 - Persistent Memory & Virtual File Systems:** Offloading large tool outputs and historical states to virtual file systems or databases to keep context windows clean.
 - Hierarchical Delegation:** Spawning specialized sub-agents or delegating tasks to specific worker nodes (such as the supervisor and subagent patterns you reviewed).
 - Durable Execution & Checkpointing:** Operating on managed runtimes that support pausing, resuming, sandboxed code execution, and human -in-the-loop oversight.

*** API / Class ***

- Interrupt ()
- Send ()
- StateGraph ()
- Command ()